Comprehensive SQL vs NoSQL Study Guide for High-Level Design (SDE-1)

This chapter-style guide explains not only the differences between SQL and NoSQL, but also why architects choose one over the other in real systems. Every section is written with interview reasoning in mind.

1. Why Databases Exist

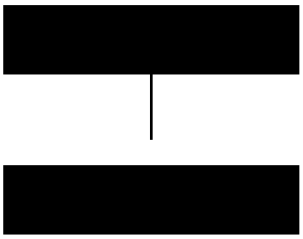
Applications must store data reliably so that it survives process restarts and machine failures. A database provides persistence, indexing, querying, concurrency control and recovery mechanisms. From an HLD perspective, the database is one of the most important architectural decisions because every request eventually interacts with data.

When traffic grows from hundreds to millions of users, storage technology directly affects scalability, latency and cost. Choosing an unsuitable database can create bottlenecks that are difficult to fix later.

Interviewers therefore expect you to justify your database selection instead of simply naming a popular technology.

2. SQL Internals

Traditional SQL Architecture



Scale Up: More CPU/RAM/SSD on one machine

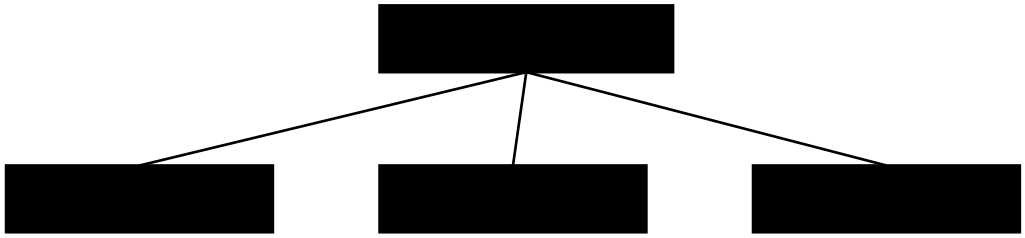
SQL databases organize data into related tables with predefined schemas. Internally they maintain indexes (commonly B+ Trees), transaction logs, query optimizers, buffer pools and storage engines. Before executing a query, the optimizer evaluates multiple execution plans and chooses an efficient one.

ACID transactions ensure correctness even when failures occur. Locks or MVCC are used to isolate concurrent transactions. WAL (Write Ahead Logging) guarantees recovery after crashes.

These characteristics make SQL ideal when correctness is more important than absolute scalability.

3. NoSQL Internals

Distributed NoSQL Architecture



Scale Out by adding new nodes

NoSQL is not a single database but a family of storage models including key-value, document, column-family and graph databases. Data is partitioned across many servers using sharding. Replication provides fault tolerance and high availability.

Most NoSQL systems optimize simple reads and writes rather than complex joins. They often denormalize data to avoid expensive distributed joins.

This architecture allows horizontal scaling to millions of users.

4. ACID vs BASE

ACID stands for Atomicity, Consistency, Isolation and Durability. It guarantees reliable transactions and is essential for banking, payments and inventory.

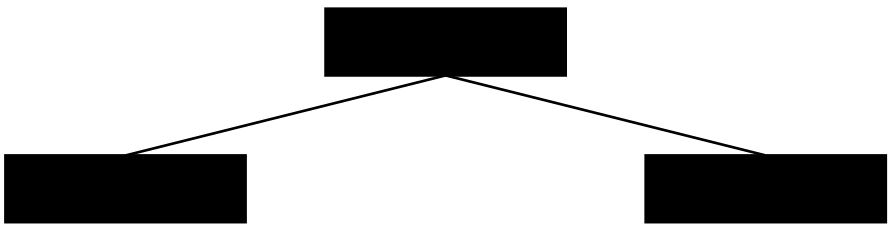
BASE stands for Basically Available, Soft State and Eventual Consistency. It accepts temporary inconsistencies to maximize availability and scalability. Social media feeds and like counters are excellent examples where BASE is acceptable.

5. CAP Theorem

CAP theorem states that during a network partition, a distributed system can choose either Consistency or Availability, but not both simultaneously. SQL databases running on a single node largely avoid this problem, whereas distributed NoSQL systems must make deliberate trade-offs.

Interviewers frequently ask why Cassandra favors availability while systems like HBase prioritize consistency.

6. Replication



Replication copies data onto multiple servers for fault tolerance. Primary-Replica replication improves read throughput while ensuring writes flow through the primary. Multi-primary replication improves write availability but increases conflict resolution complexity.

7. Sharding

Sharding horizontally partitions data across multiple machines. Users may be distributed by user ID ranges or by consistent hashing. Each shard stores only a subset of the data, allowing storage and throughput to grow linearly by adding servers.

8. Consistency Models

Strong consistency guarantees every read returns the latest committed value. Eventual consistency allows replicas to temporarily diverge before converging automatically. Read-your-own-write and causal consistency are intermediate models used by modern distributed systems.

9. Real Production Architectures

Amazon commonly uses SQL for orders, NoSQL for product catalogs and shopping carts, in-memory caches for hot data and search engines for product search.

Netflix uses distributed NoSQL databases for massive scalability and availability.

Uber stores transactional information in relational databases while location and trip-related data leverage highly scalable distributed storage.

Instagram stores social graphs, feeds and engagement data using horizontally scalable architectures because write throughput is enormous.

10. Common Mistakes

Choosing SQL only because 'it is easier', choosing NoSQL without explaining trade-offs, ignoring consistency requirements, forgetting scaling strategy, using joins in distributed NoSQL systems, and failing to justify architecture decisions are among the most common interview mistakes.

11. Revision Cheat Sheet

SQL → ACID, Joins, Fixed Schema, Strong Consistency, Vertical Scaling.
NoSQL → Flexible Schema, Horizontal Scaling, Eventual Consistency, Denormalization, Massive Throughput.

Rule of thumb:
Need correctness? SQL.
Need flexibility and huge scale? NoSQL.
Many real systems use both (Polyglot Persistence).

Frequently Asked SDE-1 Interview Questions

Why not always use NoSQL?

NoSQL improves scalability but sacrifices some relational capabilities and often consistency. Banking systems need guaranteed correctness, making SQL the better choice.

Why is shopping cart usually NoSQL?

Each cart is accessed by user ID, making key-value lookups extremely efficient while supporting millions of concurrent users.

Explain SQL vs NoSQL in one minute.

Discuss schema, scaling, transactions, consistency, joins and conclude with workload-based selection.

How does CAP relate to NoSQL?

Distributed NoSQL databases face network partitions and therefore trade consistency for availability or vice versa.

Can one application use both?

Yes. Modern architectures use polyglot persistence where each service chooses the storage engine best suited to its workload.

Difference between replication and sharding?

Replication duplicates data for availability whereas sharding partitions data for scalability.